

LEVEL 20 — Matrix Operations

NumPy examples with code, calculation, and output

1. Matrix Multiplication using *

```
import numpy as np
```

```
A = np.array([[1, 2],  
              [3, 4]])  
B = np.array([[5, 6],  
              [7, 8]])
```

```
print(A * B)
```

Output:

```
[[ 5 12]  
 [21 32]]
```

Meaning:

* performs element-by-element multiplication.
 $1 \times 5 = 5$, $2 \times 6 = 12$, $3 \times 7 = 21$, $4 \times 8 = 32$.

2. Matrix Multiplication using @

```
print(A @ B)
```

Output:

```
[[19 22]  
 [43 50]]
```

Calculation:

First value = $(1 \times 5) + (2 \times 7) = 5 + 14 = 19$

Second value = $(1 \times 6) + (2 \times 8) = 6 + 16 = 22$

Third value = $(3 \times 5) + (4 \times 7) = 15 + 28 = 43$

Fourth value = $(3 \times 6) + (4 \times 8) = 18 + 32 = 50$

3. np.dot()

```
print(np.dot(A, B))
```

Output:

```
[[19 22]  
 [43 50]]
```

For these 2-D matrices, `np.dot(A, B)` gives the same matrix multiplication result as `A @ B`.

4. np.matmul()

```
print(np.matmul(A, B))
```

Output:

```
[[19 22]  
 [43 50]]
```

`np.matmul()` performs matrix multiplication. For normal 2-D matrices, it gives the same result as

5. Quick Difference: *, @, dot(), matmul()

```
*      = element-wise multiplication
@      = matrix multiplication
np.dot = dot product / matrix multiplication depending on dimensions
np.matmul= matrix multiplication
```

For 2-D matrices:

```
A @ B == np.dot(A, B) == np.matmul(A, B)
```

6. Dot Product

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
```

```
print(np.dot(a, b))
```

Output:

32

Calculation:

$(1 \times 4) + (2 \times 5) + (3 \times 6)$

$= 4 + 10 + 18$

$= 32$

7. Inner Product

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
```

```
print(np.inner(a, b))
```

Output:

32

Calculation:

$(1 \times 4) + (2 \times 5) + (3 \times 6) = 32$

8. Outer Product

```
a = np.array([1, 2, 3])
b = np.array([4, 5])
```

```
print(np.outer(a, b))
```

Output:

$\begin{bmatrix} 4 & 5 \end{bmatrix}$

$\begin{bmatrix} 8 & 10 \end{bmatrix}$

$\begin{bmatrix} 12 & 15 \end{bmatrix}$

Calculation:

$1 \times [4, 5] = [4, 5]$

$2 \times [4, 5] = [8, 10]$

$3 \times [4, 5] = [12, 15]$

9. Transpose

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])
```

```
print(A.T)
```

Output:

```
[[1 4]
 [2 5]
 [3 6]]
```

Rows become columns.

10. Inverse

```
A = np.array([[1, 2],
              [3, 4]])
```

```
print(np.linalg.inv(A))
```

Output:

```
[[-2.   1. ]
 [ 1.5 -0.5]]
```

The inverse is a matrix A^{-1} such that:

$A \times A^{-1} = \text{Identity matrix.}$

11. Determinant

```
A = np.array([[1, 2],
              [3, 4]])
```

```
print(np.linalg.det(A))
```

Output:

```
-2.0
```

Calculation for a 2x2 matrix:

$|a \ b|$

$|c \ d| = (a \times d) - (b \times c)$

$= (1 \times 4) - (2 \times 3)$

$= 4 - 6$

$= -2$

12. Eigenvalues

```
A = np.array([[1, 2],
              [2, 1]])
```

```
values = np.linalg.eigvals(A)
```

```
print(values)
```

Output:

```
[ 3. -1.]
```

Eigenvalues are special numbers associated with a square matrix.

13. Eigenvalues and Eigenvectors

```
A = np.array([[1, 2],
              [2, 1]])

values, vectors = np.linalg.eig(A)

print("Eigenvalues:", values)
print("Eigenvectors:")
print(vectors)
```

Output:

Eigenvalues:
[3. -1.]

Eigenvectors:
[[0.70710678 -0.70710678]
 [0.70710678 0.70710678]]

The columns of the eigenvector matrix are the eigenvectors corresponding to the eigenvalues.

Important Summary

***** = element-wise multiplication

@ = matrix multiplication

np.dot() = dot product; for 2-D arrays it performs matrix multiplication

np.matmul() = matrix multiplication

np.inner() = inner product

np.outer() = outer product

A.T = transpose

np.linalg.inv() = inverse

np.linalg.det() = determinant

np.linalg.eig() = eigenvalues + eigenvectors

np.linalg.eigvals() = eigenvalues only